Merchant Intelligence Dashboard

Frontend pages, backend APIs, Razorpay data sources, ML/AI services, and outcome review

Product direction: Build a merchant-facing financial control center on top of the existing Zord backend. The dashboard should show not only whether a payment succeeded, but also what was intended, what Razorpay observed, what was settled, what reached the bank, what remains uncertain, and what the merchant should investigate next.

> **Core promise:** The dashboard turns fragmented payment, settlement, bank, CSV, and operational data into an explainable view of incoming money, outgoing money, gaps, risks, and expected cash position.

This document expands the current pages—file upload, transactions, intents, settlements, payment gaps, ML/intelligence, and outcome review—into a complete frontend information architecture with backend API contracts and response examples.

1. Product scope for a merchant

A merchant using Razorpay needs to answer six operational questions:

Question	Dashboard capability
What did I expect to collect?	Orders and payment intents.
What did Razorpay receive?	Provider payment observations and webhook/API history.
What did Razorpay settle?	Settlement and settlement-reconciliation views.
What reached my bank?	Bank statement upload/import and UTR matching.
What is missing or inconsistent?	Payment gaps, settlement gaps, exceptions, and outcome review.
What will happen next?	Incoming/outgoing cash forecast and AI/ML intelligence.

The dashboard must distinguish facts from estimates. Provider and bank observations are financial evidence. ML forecasts and AI explanations are derived intelligence and must

never overwrite the financial source of truth.

2. Recommended frontend page map

The user’ s current pages should become the following merchant navigation:

Plain Text

```
/dashboard
/data-imports
/transactions
/transactions/:id
/intents
/intents/:id
/settlements
/settlements/:id
/payment-gaps
/payment-gaps/:id
/outcome-review
/outcome-review/:id
/cash-forecast
/order-forecast
/intelligence
/ask
/connectors
/audit-evidence
/settings
```

Page priority

Priority	Pages	Build reason
P0	Dashboard, Data Imports, Transactions, Intents, Settlements, Payment Gaps, Transaction Detail	Demonstrates the core payment-to-settlement problem.
P1	Outcome Review, Intelligence, Cash Forecast, Ask Zord, Audit Evidence	Demonstrates the AI/ML and evidence differentiation.
P2	Order Forecast, Connectors, Settings, advanced dispute/refund views	Adds operational maturity after the core loop works.

3. Global frontend design rules

Every page should include the selected merchant, connector, Test/Live mode, date range, last synchronization time, and data freshness. The UI should always display the source of a value:

```
Plain Text

Source badges:
[Intent] [Razorpay Webhook] [Razorpay API] [Bank CSV] [Derived] [Forecast]
```

Every row should support a drill-down to its evidence chain. A monetary amount must include currency and should be formatted from integer currency subunits only after the backend supplies the currency.

The frontend should not call Razorpay directly. It calls the merchant backend. The backend resolves tenant, connector, secrets, provider mode, permissions, and source evidence.

4. Backend API conventions

Base URL

```
Plain Text

/api/v1/merchant
```

Common request headers

```
Plain Text

Authorization: Bearer <merchant-session-token>
X-Request-ID: <client-generated-request-id>
```

The backend derives `tenant_id` from the authenticated session. The client must not be allowed to choose an arbitrary tenant ID.

Common response envelope

```
JSON

{
  "data": {},
```

```
  "meta": {
    "request_id": "req_01J...",
    "generated_at": "2026-08-29T10:30:00Z",
    "freshness": {
      "as_of": "2026-08-29T10:25:00Z",
      "status": "fresh"
    }
  },
  "errors": []
}
```

Common error envelope

JSON

```
{
  "data": null,
  "meta": {
    "request_id": "req_01J..."
  },
  "errors": [
    {
      "code": "CONNECTOR_NOT_READY",
      "message": "Razorpay Test Mode connector requires configuration",
      "retryable": false
    }
  ]
}
```

Never return API secrets, webhook secrets, authorization headers, raw card data, or unrestricted raw Razorpay payloads to the frontend.

5. Page 1 — Merchant Dashboard

Route

Plain Text

```
/dashboard
```

Purpose

This is the merchant’ s daily control center. It summarizes incoming money, outgoing

money, settlement status, unresolved gaps, data freshness, and forecasted cash.

Widgets

Plain Text

```
1. Gross payment volume
2. Captured payments
3. Razorpay-settled amount
4. Bank-credited amount
5. Unsettled amount
6. Payment gaps
7. Settlement gaps
8. Refunds and disputes
9. Expected incoming cash next 7/30 days
10. Expected outgoing cash next 7/30 days
11. Net cash forecast
12. Top anomalies requiring review
13. Data freshness and connector health
```

API

Plain Text

```
GET /api/v1/merchant/dashboard/summary?from=2026-08-01&to=2026-08-29&currency=I
```

Response

JSON


```
{
  "data": {
    "period": {
      "from": "2026-08-01",
      "to": "2026-08-29",
      "currency": "INR"
    },
    "incoming": {
      "gross_payment_amount": 125000000,
      "captured_amount": 118000000,
      "settled_amount": 114420000,
      "bank_credited_amount": 113870000,
      "unsettled_amount": 3580000
    },
    "outgoing": {
```

```
      "refund_amount": 3200000,
      "transfer_amount": 1250000,
      "fees_and_tax": 333000
    },
    "gaps": {
      "payment_gap_count": 14,
      "settlement_gap_count": 6,
      "bank_match_gap_count": 3,
      "high_priority_count": 4
    },
    "forecast": {
      "next_7_days_incoming": 28200000,
      "next_7_days_outgoing": 9100000,
      "next_7_days_net": 19100000,
      "confidence": 0.81
    },
    "top_actions": [
      {
        "action_id": "act_001",
        "type": "review_missing_bank_credit",
        "title": "3 settlements have no matching bank credit",
        "severity": "high",
        "count": 3
      }
    ],
    "freshness": {
      "last_webhook_at": "2026-08-29T10:24:00Z",
      "last_api_backfill_at": "2026-08-29T10:25:00Z",
      "last_bank_import_at": "2026-08-28T18:00:00Z",
      "status": "fresh"
    }
  }
}
```


Backend implementation

Use the existing outcome, settlement, evidence, intelligence, and forecast services. Do not calculate dashboard totals in React. Add a read-model query in the backend so every widget uses the same period, tenant, connector, currency, and source definitions.

6. Page 2 — Data Imports

Route

Plain Text

```
/data-imports
```

Purpose

This page supports the user’ s existing CSV upload requirement for transaction files, settlement files, and bank statements. It must show upload status, column mapping, validation errors, imported counts, duplicate counts, and the resulting reconciliation impact.

Upload types

Import type	Required use
`transactions_csv`	Merchant order/payment intent records.
`settlements_csv`	Settlement report or settlement breakdown.
`bank_statement_csv`	Bank credits/debits used for settlement proof.
`refunds_csv`	Optional refund repair data.
`orders_csv`	Optional order-system data for expected payment intent.

API flow

Plain Text

```
POST /api/v1/merchant/imports/upload-url
POST /api/v1/merchant/imports
GET  /api/v1/merchant/imports
GET  /api/v1/merchant/imports/:import_id
POST /api/v1/merchant/imports/:import_id/validate
POST /api/v1/merchant/imports/:import_id/commit
```

Create upload request

JSON

```
{
  "import_type": "settlements_csv",
  "file_name": "razorpay-settlements-2026-08-29.csv",
```

```
  "content_type": "text/csv",
  "size_bytes": 24591,
  "source_label": "Razorpay Dashboard export",
  "currency": "INR"
}
```

Upload response

JSON

```
{
  "data": {
    "import_id": "imp_001",
    "upload_url": "https://storage.example/signed-url",
    "expires_at": "2026-08-29T10:35:00Z",
    "status": "awaiting_upload"
  }
}
```

Validation response

JSON

```
{
  "data": {
    "import_id": "imp_001",
    "status": "validated",
    "detected_columns": [
      "settlement_id",
      "entity_id",
      "type",
      "amount",
      "fee",
      "tax",
      "settlement_utr"
    ],
    "mapping": {
      "settlement_id": "settlement_id",
      "entity_id": "provider_entity_id",
      "amount": "gross_amount",
      "fee": "fee_amount",
      "tax": "tax_amount",
      "settlement_utr": "utr"
    },
    "preview": {
      "rows_seen": 120,
```

```
      "valid_rows": 117,
      "invalid_rows": 3,
      "duplicate_rows": 0
    },
    "errors": [
      {
        "row": 18,
        "column": "amount",
        "code": "INVALID_NUMBER",
        "message": "Amount is not numeric"
      }
    ]
  }
}
```

Commit response

JSON

```
{
  "data": {
    "import_id": "imp_001",
    "status": "committed",
    "inserted_count": 117,
    "updated_count": 0,
    "duplicate_count": 0,
    "rejected_count": 3,
    "response_hash": "sha256:...",
    "reconciliation_job_id": "recon_job_001"
  }
}
```

Frontend behavior

The page needs a dropzone, import-type selector, upload progress, column-mapping table, preview table, validation-error drawer, commit button, and import-history table. Do not commit automatically after upload; the merchant must see validation results first.

7. Page 3 — Transactions

Route

Plain Text

```
/transactions
```

Purpose

Show the provider and merchant transaction ledger, with filters for payment status, source, amount, method, order ID, provider ID, webhook/API presence, settlement state, and gap state.

Razorpay’ s Payments API supports fetching a payment by ID, listing payments, and fetching payments linked to an order. The list endpoint supports date filters and pagination using `from`, `to`, `count`, and `skip`; the documented maximum `count` is 100.[1]

API

Plain Text

```
GET /api/v1/merchant/transactions?from=...&to=...&status=captured&source=all&pa
GET /api/v1/merchant/transactions/:transaction_id
GET /api/v1/merchant/transactions/:transaction_id/evidence
GET /api/v1/merchant/transactions/export
```

List response

JSON


```
{
  "data": {
    "items": [
      {
        "transaction_id": "txn_001",
        "intent_id": "intent_001",
        "order_id": "order_001",
        "provider_payment_id": "pay_test_001",
        "amount": 100000,
        "currency": "INR",
        "status": "captured",
        "payment_method": "upi",
        "sources": ["razorpay_webhook", "razorpay_api"],
        "settlement_status": "settled",
        "bank_credit_status": "matched",
        "gap_status": "none",
        "created_at": "2026-08-29T09:00:00Z",
        "captured_at": "2026-08-29T09:00:05Z",
        "settled_at": "2026-08-30T03:00:00Z"
```

```
      }
    ],
    "summary": {
      "count": 1,
      "gross_amount": 100000,
      "captured_amount": 100000,
      "settled_amount": 100000,
      "bank_credited_amount": 100000
    }
  },
  "meta": {
    "page": 1,
    "page_size": 50,
    "total": 1
  }
}
```

Transaction detail

The detail page should display a vertical evidence timeline:

Plain Text

```
Intent created
Order created
Payment authorized
Payment captured
Webhook received
API backfill observed
Settlement recon observed
Bank credit matched
Evidence pack generated
```

API response:

JSON

```
{
  "data": {
    "transaction_id": "txn_001",
    "identity": {
      "intent_id": "intent_001",
      "order_id": "order_001",
      "provider_payment_id": "pay_test_001"
    },
    "amounts": {
      "intended": 100000,
```

```
      "captured": 100000,
      "settled_gross": 100000,
      "fee": 2900,
      "tax": 522,
      "settled_net": 96578,
      "bank_credited": 96578,
      "currency": "INR"
    },
    "state": {
      "payment": "captured",
      "provider_settlement": "settled",
      "bank_credit": "matched",
      "reconciliation": "matched"
    },
    "evidence_timeline": [],
    "source_records": [
      {
        "source": "razorpay_webhook",
        "record_id": "receipt_001",
        "observed_at": "2026-08-29T09:00:06Z",
        "raw_hash": "sha256:..."
      },
      {
        "source": "razorpay_api",
        "record_id": "resp_001",
        "observed_at": "2026-08-29T09:10:00Z",
        "raw_hash": "sha256:..."
      }
    ]
  }
}
```


8. Page 4 — Payment Intents and Orders

Routes

Plain Text

```
/intents
/intents/:intent_id
```

Purpose

This page answers what the merchant expected to collect before the provider execution. It should connect the internal order, customer/cart reference, amount, currency, payment

attempt, and final outcome.

API

Plain Text

```
GET /api/v1/merchant/intents?from=...&to=...&status=...
GET /api/v1/merchant/intents/:intent_id
GET /api/v1/merchant/intents/:intent_id/attempts
POST /api/v1/merchant/intents/import
```

Response

JSON


```
{
  "data": {
    "items": [
      {
        "intent_id": "intent_001",
        "merchant_order_reference": "WEB-10001",
        "provider_order_id": "order_001",
        "amount": 100000,
        "currency": "INR",
        "state": "paid",
        "payment_attempts": 1,
        "captured_amount": 100000,
        "settled_amount": 96578,
        "gap_status": "none",
        "created_at": "2026-08-29T08:59:00Z"
      }
    ]
  }
}
```


Detail response

JSON


```
{
  "data": {
    "intent_id": "intent_001",
    "expected": {
      "amount": 100000,
      "currency": "INR",
```

```
      "merchant_order_reference": "WEB-10001"
    },
    "execution": {
      "provider": "razorpay",
      "provider_order_id": "order_001",
      "attempts": [
        {
          "attempt_id": "attempt_001",
          "provider_payment_id": "pay_test_001",
          "status": "captured",
          "amount": 100000
        }
      ]
    },
    "outcome": {
      "captured": 100000,
      "settled_net": 96578,
      "bank_credited": 96578
    },
    "recommendation": {
      "type": "none",
      "confidence": 0.98
    }
  }
}
```


9. Page 5 — Settlements

Routes

Plain Text

```
/settlements
/settlements/:settlement_id
```

Purpose

Show settlement batches, gross amount, deductions, net amount, UTR, bank matching, settlement timeline, channel/balance type, and component breakdown.

Razorpay’ s settlement dashboard exposes settlement details, breakdowns, service fees, taxes, adjustments, transfers, refunds, and settlement timelines. Razorpay also provides a settlement-reconciliation API that returns transaction types such as payments, refunds,

transfers, and adjustments, with fields such as `entity_id`, `debit`, `credit`, `fee`, `tax`, `settlement_id`, `settlement_utr`, and `payment_id`.[2] [3]

API

Plain Text

```
GET /api/v1/merchant/settlements?from=...&to=...&status=...
GET /api/v1/merchant/settlements/:settlement_id
GET /api/v1/merchant/settlements/:settlement_id/lines
GET /api/v1/merchant/settlements/:settlement_id/evidence
```

List response

JSON


```
{
  "data": {
    "items": [
      {
        "settlement_id": "setl_001",
        "provider": "razorpay",
        "provider_mode": "test",
        "gross_amount": 5000000,
        "fee_amount": 145000,
        "tax_amount": 26100,
        "refund_amount": 100000,
        "transfer_amount": 0,
        "adjustment_amount": 0,
        "net_amount": 4713900,
        "currency": "INR",
        "settlement_utr": "utr_001",
        "status": "bank_matched",
        "settled_at": "2026-08-29T03:00:00Z",
        "bank_credit_date": "2026-08-29"
      }
    ]
  }
}
```


Detail response

JSON

```
{
  "data": {
    "settlement_id": "setl_001",
    "calculation": {
      "payment_credit": 5000000,
      "refund_debit": 100000,
      "transfer_debit": 0,
      "fee_debit": 145000,
      "tax_debit": 26100,
      "adjustments": 0,
      "expected_net": 4713900,
      "bank_credit": 4713900,
      "difference": 0,
      "currency": "INR"
    },
    "lines": [
      {
        "entity_id": "pay_test_001",
        "type": "payment",
        "credit": 100000,
        "debit": 0,
        "fee": 2900,
        "tax": 522,
        "payment_id": "pay_test_001"
      }
    ],
    "bank_match": {
      "status": "matched",
      "bank_transaction_id": "bank_txn_001",
      "utr": "utr_001",
      "matched_amount": 4713900
    }
  }
}
```


10. Page 6 — Payment Gaps

Routes

Plain Text

```
/payment-gaps
/payment-gaps/:gap_id
```

Purpose

This is the most important operational page. It displays unresolved differences across intent, payment, settlement, and bank observation.

Gap types

Plain Text

```
intent_without_payment
payment_without_intent
payment_without_webhook
payment_without_settlement
settlement_without_bank_credit
bank_credit_without_settlement
amount_mismatch
fee_tax_mismatch
duplicate_provider_record
out_of_order_event
stale_observation
schema_error
```

API

Plain Text

```
GET /api/v1/merchant/payment-gaps?severity=high&status=open
GET /api/v1/merchant/payment-gaps/:gap_id
POST /api/v1/merchant/payment-gaps/:gap_id/assign
POST /api/v1/merchant/payment-gaps/:gap_id/resolve
POST /api/v1/merchant/payment-gaps/:gap_id/request-backfill
```

List response

JSON


```
{
  "data": {
    "items": [
      {
        "gap_id": "gap_001",
        "type": "settlement_without_bank_credit",
        "severity": "high",
        "status": "open",
        "payment_id": "pay_test_001",
```

```
        "settlement_id": "setl_001",
        "expected_amount": 96578,
        "observed_amount": 0,
        "currency": "INR",
        "age_hours": 31,
        "recommended_action": "check bank statement or request settlement backf
        "confidence": 0.94,
        "created_at": "2026-08-28T03:00:00Z"
      }
    ],
    "summary": {
      "open": 14,
      "high": 4,
      "medium": 7,
      "low": 3,
      "total_amount_at_risk": 342000
    }
  }
}
```


Detail response

JSON


```
{
  "data": {
    "gap_id": "gap_001",
    "classification": {
      "type": "settlement_without_bank_credit",
      "severity": "high",
      "confidence": 0.94
    },
    "expected": {
      "settlement_net": 96578,
      "currency": "INR"
    },
    "observed": {
      "razorpay_settlement": true,
      "bank_credit": false,
      "last_bank_import_at": "2026-08-28T18:00:00Z"
    },
    "evidence": [],
    "recommended_actions": [
      {
        "action_id": "action_001",
        "label": "Run settlement and bank matching repair",
        "requires_approval": false,
```

```
        "allowed": true
      }
    ]
  }
}
```

The AI can explain a gap, but only an explicit action contract may execute a repair or state change.

11. Page 7 — Outcome Review

Routes

Plain Text

```
/outcome-review
/outcome-review/:outcome_id
```

Purpose

This page provides a human review queue for unresolved or ambiguous financial outcomes. It is where the merchant or finance operator decides whether to accept, assign, investigate, or resolve an exception.

API

Plain Text

```
GET /api/v1/merchant/outcomes/review-queue
GET /api/v1/merchant/outcomes/:outcome_id
POST /api/v1/merchant/outcomes/:outcome_id/claim
POST /api/v1/merchant/outcomes/:outcome_id/add-note
POST /api/v1/merchant/outcomes/:outcome_id/resolve
POST /api/v1/merchant/outcomes/:outcome_id/reopen
```

Response

JSON

```
{
  "data": {
    "items": [
      {
```

```
        "outcome_id": "outcome_001",
        "transaction_id": "txn_001",
        "review_reason": "amount_mismatch",
        "status": "needs_review",
        "severity": "medium",
        "expected_amount": 100000,
        "settled_amount": 96578,
        "difference": 3422,
        "currency": "INR",
        "confidence": 0.88,
        "suggested_resolution": "confirm fee and tax deduction",
        "owner": null,
        "created_at": "2026-08-29T09:10:00Z"
      }
    ]
  }
}
```


Frontend behavior

Use a three-column layout: queue, evidence/timeline, and review action panel. Resolution must require a reason. The frontend should display an immutable audit trail after resolution.

12. Page 8 — Intelligence Center

Route

Plain Text

```
/intelligence
```

Purpose

This is the combined ML and AI merchant intelligence page. It should explain historical trends, anomaly counts, payment success, settlement delays, fee leakage, refund behavior, and recommended actions.

Panels

Plain Text

```
1. Payment conversion and failure trends
2. Settlement delay distribution
```

```
3. Fee/tax variance
4. Anomaly clusters
5. Missing-webhook rate
6. Cash collection reliability
7. Top merchants/orders/channels by risk
8. AI-generated daily briefing
9. Recommended actions
```

API

Plain Text

```
GET /api/v1/merchant/intelligence/overview?from=...&to=...
GET /api/v1/merchant/intelligence/anomalies
GET /api/v1/merchant/intelligence/briefing
GET /api/v1/merchant/intelligence/recommendations
POST /api/v1/merchant/intelligence/recommendations/:id/feedback
```

Response

JSON


```
{
  "data": {
    "overview": {
      "payment_success_rate": 0.942,
      "settlement_match_rate": 0.987,
      "bank_match_rate": 0.981,
      "average_settlement_delay_hours": 19.4,
      "missing_webhook_rate": 0.012,
      "forecast_confidence": 0.81
    },
    "anomalies": [
      {
        "anomaly_id": "anom_001",
        "type": "settlement_delay",
        "description": "Settlement delay is 2.1 standard deviations above the m
        "amount_at_risk": 450000,
        "confidence": 0.86,
        "source_record_ids": ["setl_001", "setl_002"]
      }
    ],
    "briefing": {
      "generated_at": "2026-08-29T10:00:00Z",
      "text": "Collections are stable. Four settlements require review because
      "citations": [
```

```
        {
          "label": "4 unmatched settlements",
          "route": "/payment-gaps?type=settlement_without_bank_credit"
        }
      ]
    }
  }
}
```

13. Page 9 — Cash Forecast

Route

Plain Text

```
/cash-forecast
```

Purpose

Forecast money coming in, money going out, settlement timing, refunds, transfers, fees, taxes, and net cash position.

API

Plain Text

```
GET /api/v1/merchant/forecasts/cash?days=30&currency=INR
GET /api/v1/merchant/forecasts/cash/explanations
GET /api/v1/merchant/forecasts/cash/scenarios
```

Response

JSON

```
{
  "data": {
    "forecast_id": "forecast_001",
    "generated_at": "2026-08-29T10:00:00Z",
    "horizon_days": 30,
    "currency": "INR",
    "daily": [
      {
        "date": "2026-08-30",
```

```
        "incoming_expected": 4200000,
        "incoming_lower": 3500000,
        "incoming_upper": 4800000,
        "outgoing_expected": 1100000,
        "net_expected": 3100000,
        "confidence": 0.79,
        "drivers": [
          "historical weekday collections",
          "captured but unsettled payments",
          "known refund schedule"
        ]
      }
    ],
    "summary": {
      "total_incoming_expected": 92000000,
      "total_outgoing_expected": 34000000,
      "net_expected": 58000000,
      "confidence": 0.76
    },
    "model": {
      "name": "cash_forecast_v1",
      "version": "2026-08-29.1",
      "training_cutoff": "2026-08-28T23:59:59Z"
    }
  }
}
```

Forecast data must include intervals and confidence, not just one exact number. If the merchant has insufficient history, return `insufficient_history` rather than fabricate a confident forecast.

14. Page 10 — Order and Collection Forecast

Route

Plain Text

```
/order-forecast
```

Purpose

Forecast expected order volume, payment conversion, captured amount, refund amount, and settlement inflow. This is different from cash forecasting: order forecast predicts demand and collection behavior, while cash forecast predicts bank liquidity.

API

Plain Text

```
GET /api/v1/merchant/forecasts/orders?days=30
GET /api/v1/merchant/forecasts/orders/by-channel
GET /api/v1/merchant/forecasts/orders/explanations
```

Response

JSON


```
{
  "data": {
    "forecast_id": "order_forecast_001",
    "daily": [
      {
        "date": "2026-08-30",
        "expected_orders": 1420,
        "expected_paid_orders": 1331,
        "expected_captured_amount": 6800000,
        "expected_refunds": 120000,
        "conversion_rate": 0.937,
        "lower_bound": 1200,
        "upper_bound": 1650,
        "confidence": 0.73
      }
    ],
    "drivers": [
      {
        "feature": "weekday",
        "impact": 0.18,
        "direction": "positive"
      },
      {
        "feature": "recent_payment_failure_rate",
        "impact": -0.07,
        "direction": "negative"
      }
    ]
  }
}
```


15. Page 11 — Ask Zord / AI Finance Assistant

Route

Plain Text

```
/ask
```

Purpose

Allow the merchant to ask questions such as:

Plain Text

```
Why has cash received this week decreased?
Which payments were captured but not settled?
Show settlements credited by Razorpay but not found in the bank statement.
What is expected to arrive in the next seven days?
Explain this payment gap.
Which fees and taxes caused the largest settlement differences?
```

API

Plain Text

```
POST /api/v1/merchant/ask
GET  /api/v1/merchant/ask/sessions
GET  /api/v1/merchant/ask/sessions/:session_id
```

Request

JSON

```
{
  "session_id": "ask_session_001",
  "question": "Which settlements were credited by Razorpay but are missing from
  "filters": {
    "from": "2026-08-01",
    "to": "2026-08-29",
    "currency": "INR"
  }
}
```

Response

JSON


```
{
  "data": {
    "answer_id": "answer_001",
    "answer": "I found 3 Razorpay settlements marked as settled but not matched
    "confidence": 0.93,
    "answer_type": "evidence_grounded",
    "citations": [
      {
        "type": "settlement",
        "id": "setl_001",
        "label": "Settlement setl_001"
      },
      {
        "type": "gap",
        "id": "gap_001",
        "label": "Payment gap gap_001"
      }
    ],
    "suggested_actions": [
      {
        "action_id": "action_001",
        "label": "Open payment gaps",
        "route": "/payment-gaps?type=settlement_without_bank_credit",
        "requires_approval": false
      }
    ],
    "limitations": []
  }
}
```


How to tell the AI what it may do

The prompt layer should give the model structured tools, not unrestricted database access:

Plain Text

```
Tool: get_dashboard_summary
Tool: search_transactions
Tool: get_transaction_evidence
Tool: search_payment_gaps
Tool: get_settlement_breakdown
Tool: get_cash_forecast
Tool: get_order_forecast
Tool: get_import_status
```

```
Tool: create_backfill_request [approval rules]
Tool: assign_gap [approval rules]
```

The system prompt must state:

Plain Text

```
1. Financial truth comes from provider, bank, intent, and evidence records.
2. Forecasts and anomaly scores are derived, not authoritative facts.
3. Never claim a payment is bank-credited without a bank observation or explici
4. Cite every material amount and state with a record ID or dashboard link.
5. If sources disagree, describe the disagreement and create or link to a gap.
6. Never invent missing transactions, settlement dates, UTRs, or balances.
7. Do not execute financial actions directly; propose an action contract.
8. Follow tenant and user permissions.
9. Do not expose secrets or sensitive payment data.
10. If evidence is insufficient, say that the answer cannot be proven from avai
```

RAG should retrieve policies, field definitions, reconciliation rules, merchant-specific documentation, and prior resolved exception explanations. Live financial values must come from structured read tools, not vector search.

16. Page 12 — Audit Evidence

Route

Plain Text

```
/audit-evidence
```

Purpose

Show evidence packs, source hashes, event lineage, inclusion proofs, and verification state.

API

Plain Text

```
GET /api/v1/merchant/evidence
GET /api/v1/merchant/evidence/:evidence_id
GET /api/v1/merchant/evidence/:evidence_id/verify
GET /api/v1/merchant/evidence/export
```

Response

JSON


```
{
  "data": {
    "evidence_id": "evidence_001",
    "subject_type": "settlement",
    "subject_id": "setl_001",
    "status": "verified",
    "timeline": [
      {
        "event": "razorpay_api_observed",
        "source_record_id": "resp_001",
        "raw_hash": "sha256:...",
        "occurred_at": "2026-08-29T09:00:00Z"
      },
      {
        "event": "bank_csv_imported",
        "source_record_id": "bank_001",
        "raw_hash": "sha256:...",
        "occurred_at": "2026-08-29T10:00:00Z"
      }
    ],
    "merkle": {
      "root": "merkle_root_001",
      "proof_available": true
    }
  }
}
```


17. Razorpay data-source coverage

Use the following Razorpay sources in the backend. The frontend should not mirror every Razorpay Dashboard page; it should present merchant decisions derived from them.

Razorpay source	Data needed	Dashboard destination
Payments API	Payment ID, amount, currency, status, method, order ID, refund status, fee, tax, timestamps	Transactions, transaction detail, intelligence.
Order/payment relationship	Provider order and linked payments	Intents, orders, transaction detail.

Payment webhooks	Near-real-time lifecycle observations	Transaction timeline, freshness, gaps.
Settlements API	Settlement batches and timestamps	Settlements, cash forecast.
Settlement recon API	Payment/refund/transfer/adjustment lines, fee, tax, UTR	Settlement detail, payment gaps, outcome review.
Refund APIs/webhooks	Full and partial refund state	Transactions, outgoing cash, intelligence.
Disputes APIs/webhooks/reports	Dispute status and financial exposure	Exceptions, intelligence, future dispute page.
Payment Links	Link amount, expiry, partial payment, payment state	Order forecast and expected collections, if used by merchant.
Invoices/subscriptions	Recurring expected collections	Order forecast and cash forecast, if used by merchant.
Dashboard CSV reports	Historical or export-only data	Data Imports and repair workflows.

Razorpay documentation states that Payments APIs support capture/fetch operations and order-linked payment retrieval.[4] Refund APIs support full or partial refunds and fetching refunds.[5] Disputes can be viewed through APIs and Dashboard reports and can receive webhook updates.[6] Payment Links support payment collection without a website/app and can include expiry and partial-payment behavior.[7]

Do not add Payment Links, Subscriptions, Invoices, or Disputes pages to P0 unless the target merchant actually uses those products. Make the backend resource model extensible, but keep the buildathon UI focused.

18. Backend service mapping

Use the current service boundaries:

Plain Text

```
zord-edge
  - merchant API gateway
  - CSV upload authorization
  - webhook receipt queries
  - tenant and connector context
```

```
zord-outcome-engine
  - transactions and intents read models
  - payment/settlement observations
  - gap classification
  - freshness checks
  - backfill status

zord-evidence
  - timeline
  - raw hashes
  - Merkle proofs
  - verification

zord-intelligence or existing intelligence layer
  - KPIs
  - anomaly projections
  - recommendation projections

zord-ml-service
  - order forecasts
  - cash forecasts
  - anomaly scoring
  - model metadata and confidence

zord-prompt-layer
  - hybrid RAG
  - structured read tools
  - citations
  - answer safety

zord-agent-service
  - bounded actions
  - approvals
  - backfill request contracts
  - gap assignment or workflow actions

zord-airflow
  - scheduled backfill
  - freshness jobs
  - forecast refresh jobs
```

19. Frontend implementation directory

Adapt this to the actual frontend directory, but use a feature-oriented structure:

Plain Text

```
frontend/src/
├── app/router.tsx
├── layouts/MerchantLayout.tsx
├── pages/merchant/
│   ├── DashboardPage.tsx
│   ├── DataImportsPage.tsx
│   ├── TransactionsPage.tsx
│   ├── TransactionDetailPage.tsx
│   ├── IntentsPage.tsx
│   ├── IntentDetailPage.tsx
│   ├── SettlementsPage.tsx
│   ├── SettlementDetailPage.tsx
│   ├── PaymentGapsPage.tsx
│   ├── PaymentGapDetailPage.tsx
│   ├── OutcomeReviewPage.tsx
│   ├── OutcomeDetailPage.tsx
│   ├── IntelligencePage.tsx
│   ├── CashForecastPage.tsx
│   ├── OrderForecastPage.tsx
│   ├── AskPage.tsx
│   ├── AuditEvidencePage.tsx
│   └── ConnectorsPage.tsx
├── features/
│   ├── dashboard/
│   ├── imports/
│   ├── transactions/
│   ├── intents/
│   ├── settlements/
│   ├── gaps/
│   ├── outcomes/
│   ├── forecasts/
│   ├── intelligence/
│   ├── ask/
│   └── evidence/
├── api/
│   ├── merchantApi.ts
│   ├── importsApi.ts
│   ├── transactionsApi.ts
│   ├── settlementsApi.ts
│   ├── intelligenceApi.ts
│   └── forecastApi.ts
├── components/
│   ├── SourceBadge.tsx
│   ├── FreshnessBadge.tsx
│   ├── MoneyValue.tsx
│   ├── EvidenceTimeline.tsx
│   ├── ConfidenceBadge.tsx
```

```
│   ├── GapSeverityBadge.tsx
│   └── EmptyState.tsx
└── types/
    ├── merchant.ts
    ├── transaction.ts
    ├── settlement.ts
    ├── forecast.ts
    └── intelligence.ts
```

20. Frontend state requirements

Every page needs these states:

Plain Text

```
loading
success
empty
stale
partial_data
permission_denied
connector_not_ready
api_error
retrying
```

For ML/AI pages add:

Plain Text

```
forecast_not_available
insufficient_history
low_confidence
model_stale
explanation_unavailable
```

For CSV imports add:

Plain Text

```
awaiting_upload
uploading
uploaded
validating
validation_failed
validated
committing
```

```
committed
partial
failed
```

21. Recommended build order

P0 frontend

Plain Text

```
1. Merchant layout, filters, source badges, freshness badge.
2. Data Imports page and upload/validation/commit flow.
3. Transactions list and detail timeline.
4. Intents list and detail.
5. Settlements list and detail breakdown.
6. Payment Gaps list and detail.
7. Dashboard summary using the same read models.
```

P1 frontend

Plain Text

```
8. Outcome Review queue and resolution workflow.
9. Intelligence overview.
10. Cash Forecast.
11. Order Forecast.
12. Ask Zord with citations.
13. Audit Evidence.
```

P2 frontend

Plain Text

```
14. Connector health and backfill operations.
15. Refunds and disputes.
16. Payment Links, invoices, and subscriptions if merchant usage justifies them
17. Advanced scenario forecasting.
```

22. Minimum buildathon demo

The complete demo should take the judges through one merchant payment:

Plain Text

```
1. Upload a transaction or order CSV.
2. Upload a settlement CSV or pull Razorpay Test Mode data.
3. Show the transaction on the Transactions page.
4. Show the expected amount on the Intents page.
5. Show Razorpay settlement and fee/tax breakdown.
6. Intentionally include one missing bank match.
7. Show the issue on Payment Gaps.
8. Open Outcome Review and show evidence timeline.
9. Open Dashboard and show cash impact.
10. Open Intelligence and show the anomaly/recommendation.
11. Ask Ask Zord why the amount is missing.
12. Show cited answer linked to the gap and evidence.
```

23. Acceptance criteria

Plain Text

```
[ ] Every page calls the backend, never Razorpay directly.
[ ] Every financial amount has currency and source.
[ ] Dashboard totals agree with transaction and settlement read models.
[ ] Uploaded CSVs are validated before commit.
[ ] Duplicate imports do not duplicate financial records.
[ ] Transactions show intent, payment, settlement, bank, and evidence state.
[ ] Settlements show gross, fee, tax, refund, transfer, adjustment, and net.
[ ] Payment gaps show reason, amount, severity, confidence, and evidence.
[ ] Outcome review records human decisions immutably.
[ ] Forecasts show confidence and intervals.
[ ] AI answers cite structured evidence.
[ ] AI refuses unsupported claims.
[ ] Connector mode and freshness are visible.
[ ] No secret, card data, or raw authorization header reaches the frontend.
[ ] P0 works with CSV data and Razorpay Test Mode data.
```

References

[1]: https://razorpay.com/docs/api/payments/fetch-all-payments/?preferred-country=US “Razorpay Docs — Fetch All Payments”

[2]: https://razorpay.com/docs/api/settlements/fetch-recon/?preferred-country=US “Razorpay Docs — Fetch Settlement Recon Details”

[3]: https://razorpay.com/docs/payments/settlements/dashboard/ “Razorpay Docs — Settlements Dashboard Actions”

[4]: https://razorpay.com/docs/api/payments/ “Razorpay Docs — Payments APIs”

[5]: https://razorpay.com/docs/api/refunds/ “Razorpay Docs — Refund APIs”

[6]: https://razorpay.com/docs/payments/disputes/dashboard/ “Razorpay Docs — Disputes Dashboard Actions”

[7]: https://razorpay.com/docs/payments/payment-links/ “Razorpay Docs — About Payment Links”